

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA0301

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO3:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

(10 Marks)

Intelligent Traffic Signal Control Using Reinforcement Learning

The aim of this experiment is to develop an intelligent traffic signal controller using Reinforcement Learning (RL). The system learns the best traffic-light actions by interacting with a traffic simulation environment. Python, PyTorch/TensorFlow, NumPy, Pandas, Matplotlib, Gymnasium and SUMO/SUMO-RL can be used for implementation. SUMO provides a microscopic traffic simulation environment, while the RL agent observes traffic conditions and selects signal actions. SUMO-RL supports traffic-signal observations such as traffic-light phase, lane density and queue information.

Environment Design

The traffic intersection is modelled as an RL environment. The **state space** contains the current signal phase, lane vehicle density, queue length and waiting-time information. The **action space** contains discrete traffic-light decisions such as maintaining the current phase or

switching to another green phase. The reward function encourages the agent to reduce congestion:

$$\text{Reward} = - (\text{Waiting Time} + \text{Queue Length})$$

Thus, a reduction in waiting time and queue length produces a better reward.

REINFORCE and A2C

The REINFORCE algorithm is a policy-gradient method. The agent collects a complete episode of states, actions and rewards and calculates the discounted return. The policy is then updated so that actions producing higher returns become more probable. Advantage Actor-Critic (A2C) uses two components: an Actor, which selects the traffic signal action, and a Critic, which estimates the value of the current state. The advantage value is used to improve the policy update. This generally reduces the high variance associated with basic REINFORCE. Both models must be trained using the same traffic environment, traffic demand and number of episodes. Training progress can be monitored using cumulative reward and waiting-time graphs.

Basic REINFORCE Code

```
import torch
import torch.nn as nn
from torch.distributions import Categorical

class Policy(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_size, 64),
            nn.ReLU(),
            nn.Linear(64, action_size),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
        return self.net(state)

def select_action(policy, state):
    probs = policy(torch.tensor(state, dtype=torch.float32))
    dist = Categorical(probs)
    action = dist.sample()
    return action.item(), dist.log_prob(action)

def update_reinforce(optimizer, log_probs, rewards, gamma=0.99):
    returns, G = [], 0
    for r in reversed(rewards):
        G = r + gamma * G
    returns.insert(0, G)
```

```

returns = torch.tensor(returns, dtype=torch.float32)
loss = -sum(lp * G for lp, G in zip(log_probs, returns))

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

The policy interacts with the traffic environment, stores log_probs and rewards, and updates the network after each episode.

Basic A2C Code

```

class ActorCritic(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(state_size, 64),
            nn.ReLU()
        )
        self.actor = nn.Linear(64, action_size)
        self.critic = nn.Linear(64, 1)

    def forward(self, state):
        x = self.shared(state)
        return self.actor(x), self.critic(x)

```

The Actor selects the signal phase, while the Critic estimates the state value. The difference between the observed return and estimated value is used as the advantage for updating the Actor.

Evaluation

The models can be evaluated using **average reward, average waiting time, queue length, vehicle throughput and convergence speed**.

Metric	REINFORCE	A2C	Metric	REINFORCE
Learning stability	Moderate	High	Learning stability	Moderate
Training speed	Lower	Higher	Training speed	Lower
Waiting-time reduction	Good	Expected better	Waiting-time reduction	Good
Reward variance	High	Lower	Reward variance	High
Complexity	Low	Moderate	Complexity	Low

The **Episode vs Reward** graph can show whether the models learn successfully. A second graph can show **Episode vs Average Waiting Time**. A2C is expected to converge more smoothly because the Critic provides additional value information.

PPO-Based Smart HVAC Energy Management

The second experiment develops a **Proximal Policy Optimization (PPO)** model for controlling HVAC systems in a smart building. The objective is to reduce electricity consumption while maintaining occupant comfort. The environment represents a building containing rooms, occupants, temperature sensors and HVAC equipment. The **state space** can include indoor temperature, outdoor temperature, humidity, occupancy level, current HVAC setpoint, time of day and previous energy consumption. The **action** represents HVAC control decisions such as increasing, decreasing or maintaining the temperature setpoint.

The reward function combines energy efficiency and comfort:

Reward = – Energy Consumption – Comfort Penalty

When the indoor temperature remains within the desired comfort range, the comfort penalty is small. If the room becomes too hot or too cold, the penalty increases.

PPO Implementation

```
import torch
import torch.nn as nn
from torch.distributions import Categorical

class PPOActor(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_size, 64),
            nn.Tanh(),
            nn.Linear(64, action_size),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
        return self.net(state)

def ppo_loss(old_logp, new_logp, advantage,
             epsilon=0.2):
    ratio = torch.exp(new_logp - old_logp)

    clipped = torch.clamp(
        ratio,
        1 - epsilon,
        1 + epsilon
    ) * advantage

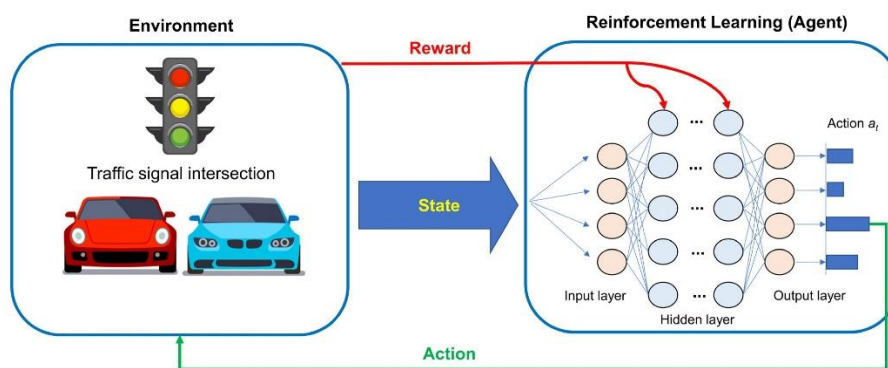
    return -torch.min(
        ratio * advantage,
        clipped
    ).mean()
```

PPO restricts the policy update using a clipping mechanism. Therefore, the new policy cannot move too far from the previous policy during a single update. This improves training stability.

PPO vs REINFORCE

The trained PPO and REINFORCE models are evaluated using **total energy consumption, average comfort deviation, cumulative reward and convergence speed.**

Metric	PPO	REINFORCE
Energy consumption	Expected lower	Moderate
Comfort maintenance	High	Moderate
Training stability	High	Lower
Convergence	Faster	Slower
Policy update	Controlled	Unrestricted



Experimental Analysis and Conclusion

The experiments demonstrate that RL can solve optimization problems involving dynamic environments. In traffic signal control, the agent learns to modify signal phases according to traffic conditions rather than using a fixed timing strategy. A2C is expected to provide more stable learning than REINFORCE, while REINFORCE provides a simpler baseline. For smart HVAC management, PPO learns a policy that balances energy consumption and occupant comfort. Its clipped objective prevents excessively large policy updates and generally provides stable training. Therefore, PPO is expected to outperform REINFORCE in the HVAC application, particularly in energy efficiency, comfort maintenance and convergence stability. The final implementation should record rewards and evaluation metrics after every episode and generate graphs such as Episode vs Reward, Episode vs Waiting Time, Episode vs Energy Consumption, and Episode vs Comfort Deviation. These graphs provide visual evidence of learning and make the comparison between algorithms clearer.

2. **Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.**
(10 Marks)

1. Objective

The objective of this experiment is to implement a Trust Region Policy Optimization (TRPO) algorithm for controlling an industrial robotic arm in an automated manufacturing environment. The robotic arm learns a policy for performing tasks such as pick-and-place, object positioning, assembly, and material handling. The model is evaluated using policy stability, movement precision, convergence speed, and overall production efficiency. A simulated environment can be developed using Python, PyTorch, Gymnasium and a robotics simulator such as PyBullet. The simulator provides the robotic arm, target objects, joint positions and physical environment, while TRPO learns the best control policy.

2. Environment Design

The robotic arm is represented as an RL agent. The **state space** contains the robot's joint positions, joint velocities, end-effector position, target position and distance between the end-effector and target. The **action space** consists of continuous joint movements or joint velocity commands. The agent observes the current robot state and generates an action to move the arm toward the required target. The reward function encourages accurate and efficient movement:

Reward = – Distance Error – Movement Cost + Task Completion Reward

A large positive reward is given when the robot successfully reaches or picks the target. Penalties are given for large positioning errors, unnecessary movement and collisions.

3. TRPO Algorithm

Trust Region Policy Optimization (TRPO) is a policy-gradient reinforcement learning algorithm designed to improve policy stability. Instead of making unrestricted policy updates, TRPO constrains the change between the old and new policies using a **KL-divergence constraint**.

The main objective can be represented as:

Maximize:

$$L(\theta) = E[\pi_{\theta}(a|s) / \pi_{old}(a|s) \times A(s,a)]$$

subject to:

$$KL(\pi_{old} || \pi_{\theta}) \leq \delta$$

where $A(s,a)$ is the advantage function and δ represents the maximum allowed policy change. This trust-region constraint prevents excessively large updates and is particularly useful for robotic control where unstable policies can result in inaccurate or unsafe movements.

4. Basic Python Implementation

The following code demonstrates the core TRPO policy structure using PyTorch.

```
import torch
import torch.nn as nn
from torch.distributions import Normal

class Policy(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh()
        )

        self.mean = nn.Linear(128, action_size)
        self.log_std = nn.Parameter(
            torch.zeros(action_size)
        )

    def forward(self, state):
        x = self.network(state)
        mean = self.mean(x)
        std = torch.exp(self.log_std)

        return Normal(mean, std)

def compute_returns(rewards, gamma=0.99):
    returns = []
    G = 0

    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)

    return torch.tensor(returns)

def policy_loss(policy, states, actions, advantages):
    dist = policy(states)
    log_prob = dist.log_prob(actions)

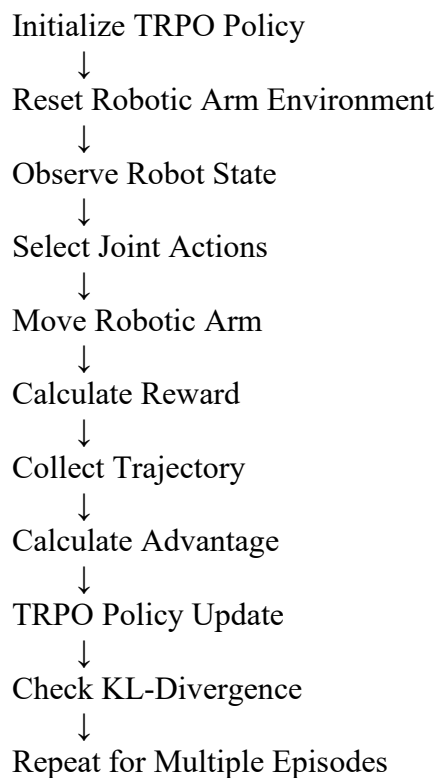
    return -(log_prob * advantages).mean()
```

The policy network receives the robot state and generates a probability distribution for continuous joint actions. During training, the robot interacts with the environment, collects rewards and calculates the advantage of each action. A complete TRPO implementation additionally calculates the Fisher information matrix, performs a conjugate-gradient search, applies a line search, and checks the KL-divergence constraint before accepting the policy update.

5. Training Process

During training, the robotic arm initially performs random or inaccurate movements. After each interaction, the agent receives a reward based on the distance from the target, movement efficiency and successful task completion.

The training procedure is:



The training progress can be monitored using **episode reward, task success rate, positioning error and episode duration**.

6. Performance Evaluation

The trained TRPO model can be evaluated using the following metrics.

Metric	Evaluation
Policy Stability	Variation in reward between episodes
Precision	Distance between end-effector and target
Convergence Speed	Number of episodes required to reach stable performance
Task Success Rate	Percentage of successfully completed tasks
Production Efficiency	Tasks completed per unit time

Policy Stability: A stable policy should produce relatively consistent rewards after convergence.

Precision: Precision can be measured using the final distance between the robotic arm's end-effector and the target. A smaller distance indicates better positioning accuracy.

Convergence Speed: The number of training episodes required for the reward to become stable is used to evaluate convergence.

Production Efficiency: Production efficiency can be measured using the number of successfully completed manufacturing tasks per minute while considering movement time and failures.

7. Expected Experimental Results

A results table can be prepared after running the simulation.

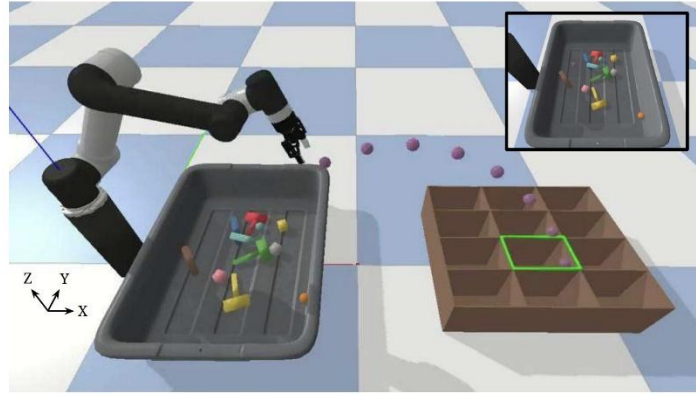
Metric	TRPO Result
Average Reward	High after convergence
Positioning Error	Low
Task Success Rate	High
Policy Stability	High
Convergence	Stable
Production Efficiency	Improved

The most important graphs are **Episode vs Reward**, **Episode vs Positioning Error**, and **Episode vs Task Success Rate**. A steadily increasing reward indicates that the robot is learning a better control policy. A decreasing positioning error demonstrates improvement in movement precision.

8. Analysis

TRPO is particularly suitable for robotic control because it restricts policy updates using a trust-region constraint. This reduces the possibility of sudden policy changes and can result in more stable learning. In an industrial manufacturing environment, stable control is important because inaccurate movements can cause failed operations, product damage or unnecessary production delays.

The main limitation of TRPO is its computational complexity. Calculating the curvature information and performing conjugate-gradient and line-search operations makes it more complicated than simpler policy-gradient algorithms. Training can therefore require more computational resources.



9. Conclusion

The TRPO-based robotic-arm system provides a reinforcement-learning approach for automated manufacturing. By observing robot and target positions and receiving rewards based on precision and task completion, the agent learns an effective control policy. TRPO's trust-region constraint improves policy stability by preventing excessively large updates. The final system can be evaluated using **precision, stability, convergence speed, task success rate, collision rate and production efficiency**. A successful implementation should demonstrate increasing cumulative reward, decreasing positioning error and improved task-completion efficiency as training progresses.

3. Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

(10 Marks)

1. Objective

The objective of this experiment is to design and develop a Policy-Based Reinforcement Learning system for adaptive patient treatment planning. The system learns treatment decisions using simulated patient data and compares Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO) algorithms. The comparison is performed using treatment effectiveness, cumulative reward and learning stability. The experiment uses simulated patient data, so the model does not make decisions for real patients. Python, PyTorch, NumPy, Pandas and Matplotlib can be used for implementation.

2. RL Environment Design

The patient is represented as an RL environment. At each treatment step, the agent observes the patient's current condition and selects an appropriate treatment action.

The **state space** contains simulated values such as:

- Patient age group
- Disease severity
- Symptom score
- Previous treatment response
- Vital-sign indicators
- Recovery progress

The **action space** represents different simulated treatment strategies, such as Treatment A, Treatment B and Treatment C.

The reward function measures improvement in the patient's simulated condition:

Reward = Improvement in Health Score – Treatment Cost – Side-Effect Penalty

A higher reward represents better simulated treatment outcomes with fewer negative effects.

3. Simulated Patient Data

A simple simulated dataset can be generated using Python. Each patient is assigned a health score, severity level and treatment-response value.

```
import numpy as np
import pandas as pd
```

```
np.random.seed(42)
```

```

patients = pd.DataFrame({
    "severity": np.random.randint(1, 6, 200),
    "symptom_score": np.random.randint(1, 11, 200),
    "previous_response": np.random.uniform(0, 1, 200),
    "health_score": np.random.uniform(40, 90, 200)
})

print(patients.head())

```

The dataset contains 200 simulated patient records. These values are only used to demonstrate the RL methodology.

4. A2C Implementation

A2C uses an **Actor** to select the treatment action and a **Critic** to estimate the value of the patient's current state.

```

import torch
import torch.nn as nn

class A2C(nn.Module):
    def __init__(self, state_size, action_size):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(state_size, 64),
            nn.ReLU()
        )

        self.actor = nn.Linear(64, action_size)
        self.critic = nn.Linear(64, 1)

    def forward(self, state):
        x = self.shared(state)

        policy = torch.softmax(
            self.actor(x), dim=-1
        )

        value = self.critic(x)

        return policy, value

```

During training, the Actor selects a treatment action and receives a reward from the simulated environment. The Critic estimates the expected future reward. The advantage is calculated as the difference between the actual return and predicted value.

5. PPO Implementation

PPO also uses a policy network but restricts the size of policy updates. This prevents the model from making excessively large changes during training.

```
def ppo_loss(old_log_prob,
            new_log_prob,
            advantage,
            epsilon=0.2):

    ratio = torch.exp(
        new_log_prob - old_log_prob
    )

    clipped_ratio = torch.clamp(
        ratio,
        1 - epsilon,
        1 + epsilon
    )

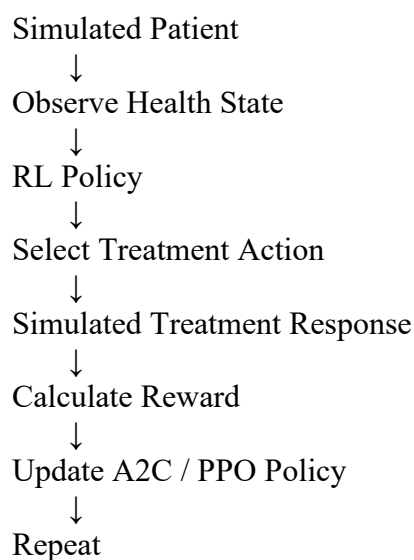
    loss = -torch.min(
        ratio * advantage,
        clipped_ratio * advantage
    )

    return loss.mean()
```

The clipping mechanism is the main feature of PPO. It helps maintain stable policy improvement while learning treatment strategies.

6. Training Process

Both A2C and PPO are trained using the **same simulated patient environment, patient dataset, number of episodes and reward function.**



During training, the following values should be recorded:

Cumulative reward → Treatment effectiveness → Learning stability → Episode performance

7. Performance Comparison

The models can be evaluated using treatment effectiveness, cumulative reward and learning stability.

Metric	A2C	PPO
Treatment effectiveness	Good	Expected higher
Cumulative reward	High	Expected higher
Learning stability	Good	High
Policy update	Actor-Critic	Clipped policy
Training complexity	Moderate	Moderate–High

Treatment effectiveness can be measured using the percentage improvement in the simulated patient health score.

Cumulative reward represents the total reward obtained by the agent during an episode.

Learning stability can be measured by observing the variation in rewards between consecutive training episodes. A smoother reward curve indicates more stable learning.

8. Graphical Analysis

Three graphs should be generated after training:

1. Episode vs Cumulative Reward

The graph compares the learning progress of A2C and PPO. Increasing reward indicates that the algorithms are learning better treatment policies.

2. Episode vs Treatment Effectiveness

This graph shows the improvement in simulated patient health scores over training episodes.

3. Episode vs Reward Stability

A PPO curve with smaller fluctuations indicates more stable learning.

Example plotting code:

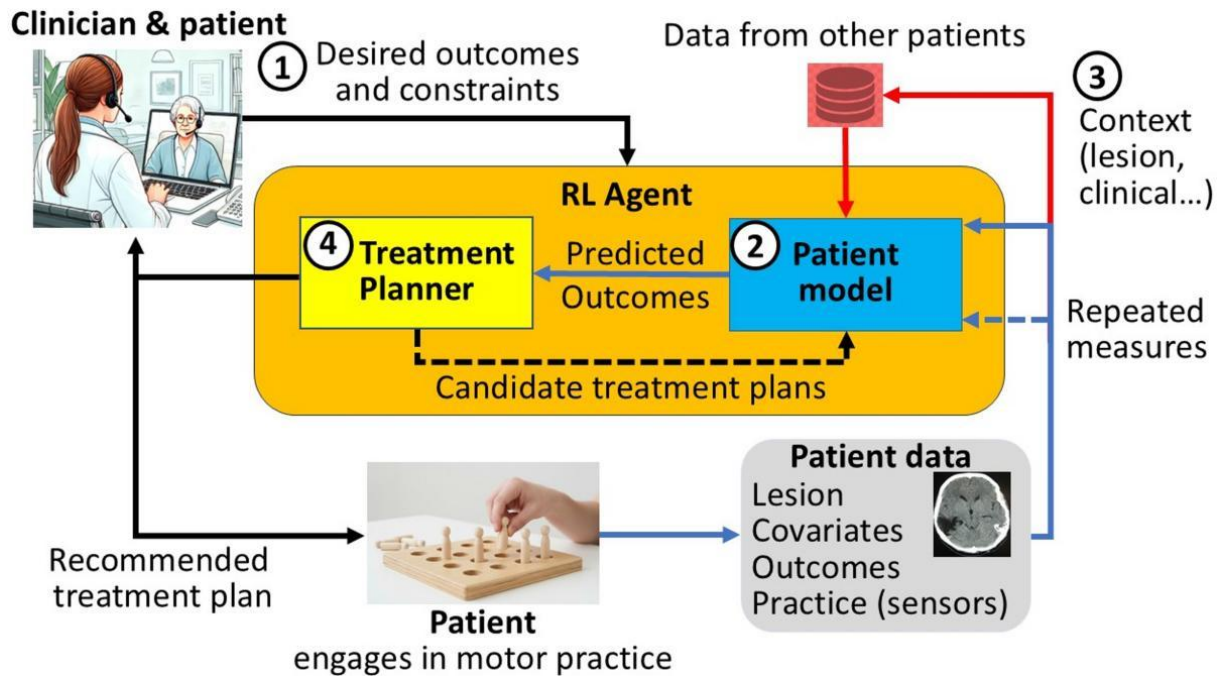
```
import matplotlib.pyplot as plt

episodes = range(1, 101)

plt.plot(episodes, a2c_rewards,
         label="A2C")
```

```
plt.plot(epochs, ppo_rewards,
        label="PPO")
```

```
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("A2C vs PPO Learning Performance")
plt.legend()
plt.show()
```



9. Analysis and Conclusion

A2C provides an effective policy-based approach because its Actor selects treatment actions while the Critic evaluates the quality of those decisions. However, its policy updates can sometimes produce larger variations during training. PPO improves policy stability by restricting the change between the old and new policies. Therefore, PPO is expected to produce **more stable learning, higher cumulative rewards and better simulated treatment effectiveness** than A2C in this experiment. The experiment demonstrates how policy-based reinforcement learning can be used to study adaptive treatment planning using simulated data. However, the results should **not be interpreted as clinical recommendations** because real healthcare systems require validated clinical datasets, medical supervision, safety constraints, ethical approval and extensive external validation. Overall, the comparison is expected to show that **PPO provides a stronger balance between treatment effectiveness, cumulative reward and learning stability**, while A2C provides a simpler and useful baseline for comparison.

4. Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

(10 Marks)

1. Objective

The objective of this experiment is to develop a Deep Reinforcement Learning (DRL) model using Proximal Policy Optimization (PPO) for autonomous drone navigation in dynamic environments. The drone learns to move from a starting position to a target location while avoiding static and moving obstacles. The model is evaluated using navigation accuracy, obstacle avoidance, energy consumption and policy convergence. The system can be developed using Python, PyTorch, NumPy, Matplotlib and Gymnasium. A simulator such as PyBullet or AirSim can be used to create the drone environment.

2. Drone Navigation Environment

The drone is represented as an RL agent. At every time step, the agent observes the environment and selects a movement action.

The **state space** can contain the drone's:

- X, Y and Z coordinates
- Velocity
- Distance from target
- Distance from nearby obstacles
- Direction toward the target

The **action space** contains continuous movement commands such as changes in velocity or movement along the X, Y and Z axes. The reward function encourages the drone to reach the target safely:

Reward = Target Progress – Collision Penalty – Energy Cost

A positive reward is given when the drone moves closer to the destination. A large penalty is applied when the drone collides with an obstacle, while a smaller penalty represents energy consumption.

3. PPO Model

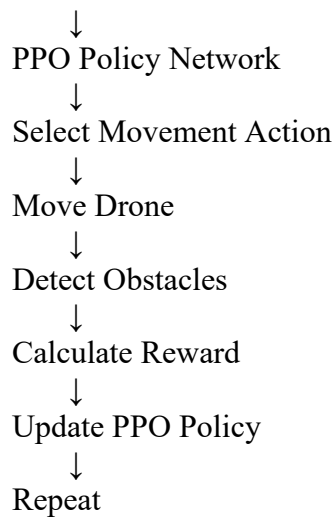
PPO is selected because it works well with continuous control problems and provides stable policy updates. The neural network receives the drone's current state and produces an action distribution.

The training process is:

Drone Environment

↓

Observe Drone State



4. Basic Python Implementation

The following code demonstrates the basic PPO policy network for drone control.

```
import torch
import torch.nn as nn
from torch.distributions import Normal

class PPOActor(nn.Module):

    def __init__(self, state_size, action_size):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh()
        )

        self.mean = nn.Linear(128, action_size)
        self.log_std = nn.Parameter(
            torch.zeros(action_size)
        )

    def forward(self, state):

        x = self.network(state)

        mean = self.mean(x)
        std = torch.exp(self.log_std)

        return Normal(mean, std)
```

```

def ppo_loss(old_logp,
             new_logp,
             advantage,
             epsilon=0.2):

    ratio = torch.exp(
        new_logp - old_logp
    )

    clipped = torch.clamp(
        ratio,
        1 - epsilon,
        1 + epsilon
    )

    loss = -torch.min(
        ratio * advantage,
        clipped * advantage
    )

    return loss.mean()

```

The actor network receives the drone state and produces continuous movement commands. PPO compares the old and new policies and clips the policy ratio to prevent excessively large updates.

5. Simple Drone Environment

A simplified environment can be created for testing the algorithm before connecting it to a complete simulator.

```

import numpy as np

class DroneEnvironment:

    def __init__(self):
        self.position = np.array(
            [0.0, 0.0, 0.0]
        )

        self.target = np.array(
            [10.0, 10.0, 5.0]
        )

    def reset(self):
        self.position = np.array(
            [0.0, 0.0, 0.0]
        )
        return self.position

```

```

def step(self, action):

    old_distance = np.linalg.norm(
        self.target - self.position
    )

    self.position += action

    new_distance = np.linalg.norm(
        self.target - self.position
    )

    reward = old_distance - new_distance

    done = new_distance < 0.5

    if done:
        reward += 100

    return self.position, reward, done

```

For a complete experiment, obstacle coordinates and moving obstacles can be added to the environment. Collision detection can then provide a large negative reward.

6. Training and Evaluation

The PPO agent is trained for multiple episodes. During every episode, the drone starts from a fixed or randomly selected starting position and attempts to reach the destination.

The following metrics are recorded:

Navigation Accuracy: Distance between the final drone position and target position.

Obstacle Avoidance: Number of collisions or minimum distance maintained from obstacles.

Energy Consumption: Total movement effort required to complete the navigation task.

Policy Convergence: Number of episodes required for the reward to become stable.

A results table can be prepared as follows:

Metric	PPO
Navigation accuracy	High
Obstacle avoidance	High
Energy consumption	Low/Moderate
Policy convergence	Stable

7. Graphical Analysis

The following graphs should be generated after training:

1. Episode vs Cumulative Reward

An increasing reward curve indicates that the drone is learning an effective navigation policy.

2. Episode vs Navigation Error

The distance between the drone and target should decrease as training progresses.

3. Episode vs Energy Consumption

A decreasing energy requirement indicates that the drone is learning efficient movement.

4. Episode vs Collision Rate

The number of collisions should decrease as the policy learns to avoid obstacles.

Example plotting code:

```
import matplotlib.pyplot as plt

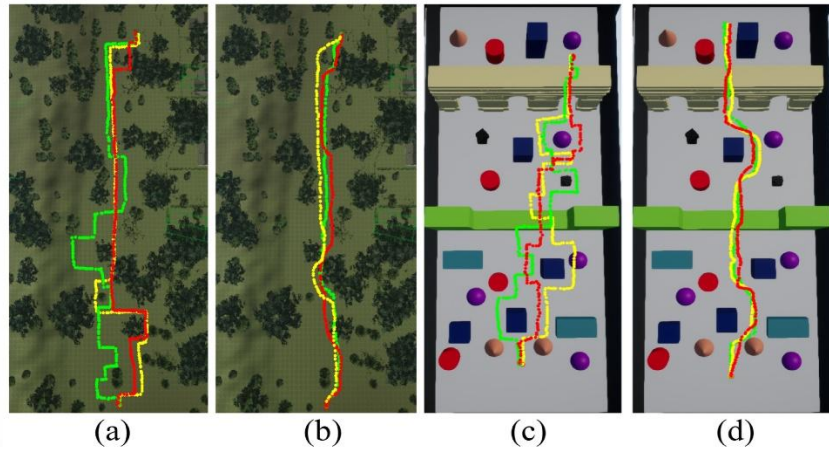
episodes = range(1, 101)

plt.plot(
    episodes,
    rewards
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("PPO Drone Navigation Learning")
plt.show()
```

8. Analysis

PPO is suitable for autonomous drone navigation because it can handle continuous control and provides stable policy updates. Initially, the drone may move randomly and frequently approach obstacles. With training, it learns to move toward the target while maintaining safer distances from obstacles. A successful model should show increasing cumulative reward, decreasing navigation error, fewer collisions and stable policy convergence. Energy consumption should also decrease as the drone learns shorter and smoother trajectories. One limitation is that simulated environments may not completely represent real-world conditions. Wind, sensor noise, battery limitations, communication delays and unexpected obstacles must be considered before deploying such a system on a real drone.



9. Conclusion

The experiment demonstrates the application of Deep Reinforcement Learning for autonomous drone navigation. A PPO agent observes the drone's position, velocity, target distance and obstacle information and learns suitable movement actions through rewards. The trained model can be evaluated using navigation accuracy, obstacle avoidance, energy consumption and convergence. Overall, PPO can provide a stable and effective navigation policy for dynamic environments. The final performance should be demonstrated using reward, navigation-error, energy-consumption and collision-rate graphs. The same framework can later be extended to more realistic simulators such as AirSim or PyBullet and tested with dynamic obstacles and environmental disturbances.

5. **Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.**
(10 Marks)

1. Objective

The objective of this experiment is to design and implement an **Asynchronous Advantage Actor-Critic (A3C)** based Reinforcement Learning framework for dynamic cloud resource allocation. The system dynamically allocates CPU and memory resources to cloud workloads according to changing demand. The performance of A3C is compared with **Vanilla Policy Gradient (VPG)** using response time, CPU utilization and resource efficiency. Python, PyTorch, NumPy, Pandas and Matplotlib can be used to implement the system. A simulated cloud environment is used instead of a real cloud infrastructure.

2. Cloud Resource Allocation Environment

The cloud platform is represented as an RL environment. At every time step, the agent observes the current workload and available resources and decides how much CPU and memory should be allocated. The **state space** contains CPU utilization, memory utilization, number of active requests, workload intensity and available resources. The **action space** consists of resource-allocation decisions such as increasing, maintaining or decreasing CPU and memory allocation. The reward function encourages low response time and efficient resource usage:

Reward = – Response Time – Resource Wastage

A high reward is obtained when workloads receive sufficient resources with minimum response time and minimum resource wastage.

3. A3C Algorithm

A3C uses multiple worker agents that interact with separate copies of the cloud environment. Each worker calculates gradients using its experience and asynchronously updates a shared global policy.

The main components are:

Actor: Selects the resource-allocation action.

Critic: Estimates the value of the current cloud state.

Workers: Explore different workload conditions simultaneously.

This parallel learning approach can improve exploration and training speed.

Basic A3C Network

```
import torch
import torch.nn as nn
```

```

class A3CNetwork(nn.Module):

    def __init__(self, state_size, action_size):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU()
        )

        self.actor = nn.Linear(
            128, action_size
        )

        self.critic = nn.Linear(
            128, 1
        )

    def forward(self, state):

        x = self.shared(state)

        policy = torch.softmax(
            self.actor(x), dim=-1
        )

        value = self.critic(x)

        return policy, value

```

The Actor produces probabilities for resource-allocation actions, while the Critic estimates the expected future reward.

4. Vanilla Policy Gradient Implementation

Vanilla Policy Gradient directly updates the policy according to the rewards received from the environment.

```

def compute_returns(rewards, gamma=0.99):

    returns = []
    G = 0

    for reward in reversed(rewards):

        G = reward + gamma * G

        returns.insert(0, G)

```

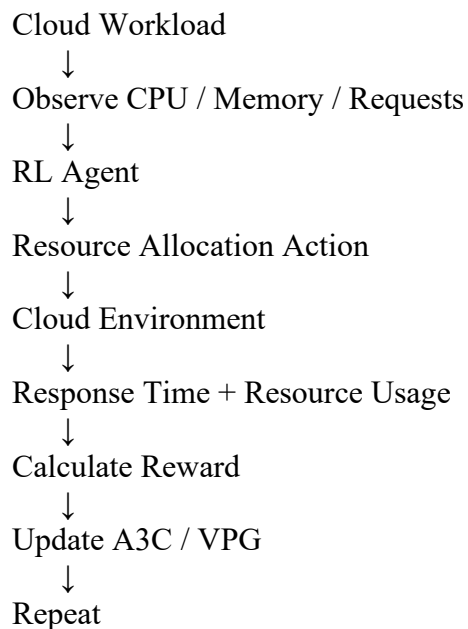
```
return returns
```

```
def policy_gradient_loss(  
    log_probs, rewards):  
  
    returns = compute_returns(rewards)  
  
    loss = 0  
  
    for log_prob, G in zip(  
        log_probs, returns):  
  
        loss += -log_prob * G  
  
    return loss.mean()
```

VPG is simpler than A3C, but it can have high variance because it learns directly from complete trajectories without the advantage estimation and parallel workers used by A3C.

5. Training Process

Both algorithms are trained using the **same simulated cloud environment, workload patterns, resource limits and number of episodes**.



For A3C, several workers interact with the environment simultaneously and update a shared global network. Training progress is recorded after each episode.

6. Performance Evaluation

The algorithms are evaluated using three major metrics.

Response Time: Average time required to process a cloud request. Lower response time indicates better performance.

CPU Utilization: Percentage of available CPU capacity being effectively used.

Resource Efficiency: Ratio of useful resource consumption to total allocated resources. Higher efficiency indicates less resource wastage.

Additional metrics such as throughput, memory utilization and cumulative reward can also be recorded.

Metric	A3C	Vanilla Policy Gradient
Response time	Expected lower	Higher
CPU utilization	Better balanced	Moderate
Resource efficiency	High	Moderate
Learning stability	High	Lower
Training speed	Faster	Slower
Exploration	Parallel	Single trajectory

7. Graphical Analysis

The following graphs should be generated after training:

1. Episode vs Cumulative Reward

Shows whether the algorithms learn increasingly effective resource-allocation policies.

2. Episode vs Response Time

A decreasing curve indicates that the agent is learning to allocate resources more effectively.

3. Episode vs CPU Utilization

Shows whether CPU resources are being used efficiently without excessive over-allocation.

4. Episode vs Resource Efficiency

A higher value indicates that the algorithm reduces unnecessary resource allocation.

Example plotting code:

```
import matplotlib.pyplot as plt
```

```
episodes = range(1, 101)
```

```
plt.plot(
    episodes,
    a3c_rewards,
    label="A3C"
)
```

```

plt.plot(
    episodes,
    vpg_rewards,
    label="Vanilla PG"
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("A3C vs Vanilla Policy Gradient")
plt.legend()
plt.show()

```

8. Experimental Analysis

A3C is expected to provide better performance because multiple workers explore different cloud workload conditions simultaneously. The Actor-Critic structure also provides an advantage estimate that can reduce the variance of policy updates.

Vanilla Policy Gradient is easier to implement but generally requires more training samples and can produce unstable updates. Under changing cloud workloads, A3C can adapt resource allocation more effectively and maintain lower response times.

A successful experiment should demonstrate **lower average response time, balanced CPU utilization, higher resource efficiency and smoother reward convergence for A3C.**

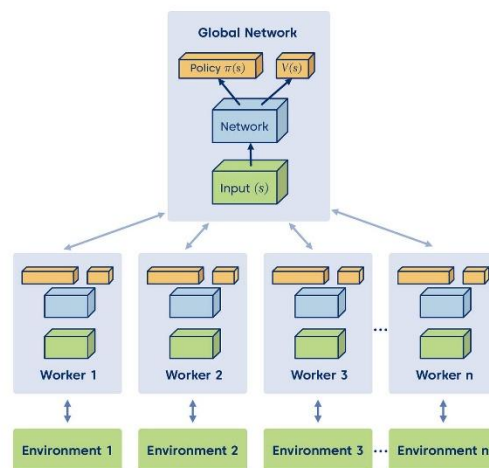


Diagram of A3C high-level architecture.

© KAPORE

9. Conclusion

The proposed A3C framework provides an intelligent approach for dynamic cloud resource allocation. Instead of using fixed resource-allocation rules, the RL agent learns to allocate resources according to workload conditions. A3C combines Actor-Critic learning with asynchronous workers, allowing efficient exploration of different resource-demand scenarios.

The comparison with Vanilla Policy Gradient demonstrates the advantages of A3C in terms of **response time, CPU utilization, resource efficiency and learning stability**. The final results can be presented using reward, response-time, CPU-utilization and resource-efficiency graphs. Overall, A3C is expected to provide a more adaptive and efficient solution for dynamic cloud resource management.

Code of Conduct:

*I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Malli Chandana

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation